70240193 Algorithms and Its Complexity Theory March 3, 2026

Randomized Algorithms Problem Set

Instructor: Jing Chen *Le Gai 2025312381*

1. Chapter 13, Problem 1: 3-Coloring

For every node $v \in V$ in graph $G = (V, E)$, we assign it a color chosen independently and uniformly at random from three colors. The time complexity of this algorithm is clearly $O(|V|)$, which is a polynomial-time algorithm.

Proof: For any edge $e = (u, v) \in E$ in the graph, we define an indicator random variable X_e as follows: If edge e is satisfied (i.e., u and v have different colors), then $X_e = 1$; otherwise $X_e = 0$.

Since the color of each node is assigned independently at random, the probability that nodes u and v receive the same color is:

$$P(\text{color}(u) = \text{color}(v)) = 3 \times \left(\frac{1}{3}\right) \times \left(\frac{1}{3}\right) = \frac{1}{3}$$

Therefore, the probability that edge e is satisfied is:

$$P(X_e = 1) = 1 - \frac{1}{3} = \frac{2}{3}$$

From this, we know the expected value of X_e is $E[X_e] = \frac{2}{3}$.

Let the random variable X denote the total number of satisfied edges, i.e., $X = \sum_{e \in E} X_e$. According to the Linearity of Expectation, we have:

$$E[X] = E\left[\sum_{e \in E} X_e\right] = \sum_{e \in E} E[X_e] = \sum_{e \in E} \frac{2}{3} = \frac{2}{3}|E|$$

Assume the optimal 3-coloring scheme satisfies c^* edges. Since the total number of edges in the graph is $|E|$, it is obvious that $c^* \leq |E|$. Thus, the expected number of edges satisfied by this algorithm is:

$$E[X] = \frac{2}{3}|E| \geq \frac{2}{3}c^*$$

This proves that the randomized algorithm can satisfy at least $\frac{2}{3}c^*$ edges in expectation.

2. Chapter 13, Problem 3: Independent Set

(a) Proof of conflict-freeness and calculation of expected size

Proof of Conflict-freeness: Assume the generated set S has a conflict, meaning there exist two adjacent processes (with a conflicting edge) P_u and P_v, and they are simultaneously selected into set S. According to the protocol rules, if P_u enters S, it must have flipped $x_u = 1$, and all processes conflicting with it must have flipped $x = 0$. However, this creates a direct contradiction with the premise that P_v also entered set S (which means P_v must have flipped $x_v = 1$). Therefore, any two adjacent processes can never simultaneously enter set S, and the resulting set S must be conflict-free.

Expected size formula: For any process P_i, define an indicator random variable I_i such that $I_i = 1$ if $P_i \in S$, and $I_i = 0$ otherwise. The necessary and sufficient condition for process P_i to enter S is that $x_i = 1$, and the x values of all its d neighbors are 0. Since the coin flips of all processes are independent:

$$P(I_i = 1) = P(x_i = 1) \prod_{j \in N(i)} P(x_j = 0) = \frac{1}{2}\left(\frac{1}{2}\right)^d = \frac{1}{2^{d+1}}$$

Thus, the expected size of set S is:

$$E[|S|] = \sum_{i=1}^{n} E[I_i] = n \cdot \frac{1}{2^{d+1}} = \frac{n}{2^{d+1}}$$

(b) Finding the optimal parameter p

Now, the protocol is modified to take 1 with probability p and 0 with probability $1 - p$. The probability that process P_i enters set S becomes:

$$P(I_i = 1) = p(1-p)^d$$

So the expected size is $E[|S|] = np(1-p)^d$. To maximize this expected value, we take the derivative of the function $f(p) = p(1-p)^d$ with respect to p and set the derivative to 0:

$$f'(p) = (1-p)^d - dp(1-p)^{d-1} = 0$$

Factoring out the common term $(1-p)^{d-1}$ (considering $p < 1$):

$$(1-p)^{d-1}[(1-p) - dp] = 0 \implies 1 - p - dp = 0 \implies p = \frac{1}{d+1}$$

Since the second derivative is negative at $p = \frac{1}{d+1}$, this point is a local maximum. The optimal probability value is $p = \frac{1}{d+1}$.

Substituting this into the expectation formula yields the optimal expected size of S:

$$E[|S|] = n\left(\frac{1}{d+1}\right)\left(1 - \frac{1}{d+1}\right)^d = \frac{n}{d+1}\left(\frac{d}{d+1}\right)^d$$

3. Chapter 13, Problem 4: Peer-to-peer

(a) Expected analysis of node in-degree

For a given node v_j and every subsequently joined node v_k ($k > j$), define an indicator random variable X_k: if v_k points to v_j, then $X_k = 1$, otherwise $X_k = 0$. Node v_k uniformly and randomly chooses one of the $k-1$ preceding nodes to establish a connection, so the probability that it points to v_j is:

$$P(X_k = 1) = \frac{1}{k-1}$$

The in-degree L_j of node v_j is the sum of all connections from $k > j$, i.e., $L_j = \sum_{k=j+1}^{n} X_k$. The expected value is:

$$E[L_j] = \sum_{k=j+1}^{n} E[X_k] = \sum_{k=j+1}^{n} \frac{1}{k-1} = \sum_{i=j}^{n-1} \frac{1}{i}$$

Exact formula: Using Harmonic numbers ($H_m = \sum_{i=1}^{m} \frac{1}{i}$), we can rewrite this as:

$$E[L_j] = H_{n-1} - H_{j-1}$$

Asymptotic expression: Since $H_m \approx \ln m + \gamma$, it can be asymptotically expressed as:

$$E[L_j] \approx \ln(n-1) - \ln(j-1) = \ln\left(\frac{n-1}{j-1}\right)$$

Expressed using Big-Θ notation, it is: $\Theta\left(\log \frac{n}{j}\right)$.

(b) Expected analysis of nodes with zero in-degree

For each node v_j $(1 \leq j \leq n)$, define an indicator random variable Y_j: if v_j has no in-degree, then $Y_j = 1$, otherwise $Y_j = 0$. Node v_j having no in-degree means that for all $k > j$, node v_k did not choose v_j. Since the choice of each node is independent:

$$P(Y_j = 1) = \prod_{k=j+1}^{n} P(v_k \not\to v_j) = \prod_{k=j+1}^{n} \left(1 - \frac{1}{k-1}\right) = \prod_{k=j+1}^{n} \frac{k-2}{k-1}$$

Expanding this, we get a telescoping product:

$$P(Y_j = 1) = \frac{j-1}{j} \cdot \frac{j}{j+1} \cdot \frac{j+1}{j+2} \cdots \frac{n-2}{n-1} = \frac{j-1}{n-1}$$

Let Y be the total number of nodes with zero in-degree, $Y = \sum_{j=1}^{n} Y_j$. The sum of the expectations for all nodes is:

$$E[Y] = \sum_{j=1}^{n} E[Y_j] = \sum_{j=1}^{n} \frac{j-1}{n-1} = \frac{1}{n-1} \sum_{j=1}^{n} (j-1)$$

The summation part is a simple arithmetic progression from 0 to $n-1$: $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$. Substituting back into the formula yields the total expectation:

$$E[Y] = \frac{1}{n-1} \cdot \frac{n(n-1)}{2} = \frac{n}{2}$$

Therefore, in this network growth model, exactly half of the nodes are expected to have no incoming connections.

4. Chapter 13, Problem 7: MAX SAT

(a) Prove that the expected number of satisfied clauses is at least $k/2$

In this completely random assignment scheme, each variable is independently set to true or false with a probability of $1/2$. For any arbitrary clause C_j consisting of $\ell_j \geq 1$ distinct variables, for the clause to be unsatisfied, all its ℓ_j variables must be assigned

the "wrong" value. Therefore, the probability that it is not satisfied is $(1/2)^{\ell_j}$. Since $\ell_j \geq 1$, the probability that clause C_j is satisfied is:

$$P(C_j \text{ is satisfied}) = 1 - \left(\frac{1}{2}\right)^{\ell_j} \geq 1 - \frac{1}{2} = \frac{1}{2}$$

By the linearity of expectation, the expected total number of satisfied clauses is:

$$E[\text{number of satisfied clauses}] = \sum_{j=1}^{k} P(C_j \text{ is satisfied}) \geq \sum_{j=1}^{k} \frac{1}{2} = \frac{k}{2}$$

Counterexample: Consider the case with exactly two clauses: $C_1 = \{x_1\}$ and $C_2 = \{\overline{x_1}\}$. Obviously, no matter what value the variable x_1 takes, at most 1 clause can be satisfied. The maximum number of satisfied clauses in this case is 1, which is exactly half of the total number of clauses ($k/2 = 1$).

(b) 0.6-approximation without conflicting single-variable clauses
Algorithm modification: For any variable appearing in a single-variable clause (e.g., $\{x_i\}$ or $\{\overline{x_i}\}$), since we know there are no conflicts, we set it to the truth value that satisfies this single-variable clause with a probability of $p = 0.6$. For all variables that do not appear in any single-variable clause, they are still randomly assigned with a probability of 0.5.
Analysis:

- **Clauses of length 1**: According to our biased setting, it is directly satisfied with a probability of 0.6.
- **Clauses of length** $c \geq 2$: In the worst-case scenario, all c variables in this clause might appear in other single-variable clauses, and the biased probability makes them lean towards leaving the current clause unsatisfied with a probability of 0.6. This means each variable only has a 0.4 probability of helping the current clause. Even under such worst-case conditions, since the variables are independent, the maximum probability that the clause is not satisfied is 0.6^c. Therefore, the probability that it is satisfied is at least $1 - 0.6^c$. Since $c \geq 2$, the probability of satisfaction is at least $1 - 0.6^2 = 1 - 0.36 = 0.64 > 0.6$.

In summary, regardless of the clause length, its probability of being satisfied is strictly no less than 0.6. Therefore, the expected total number of satisfied clauses is at least $0.6k$.

(c) 0.6 expected approximation algorithm for general MAX SAT
For the general case where conflicting single-variable clauses exist, we use a randomized rounding algorithm based on Linear Programming (LP) relaxation.

Algorithm: Express MAX SAT as an integer program, then relax it to a linear program. Maximize $\sum z_j$, subject to for each clause C_j:

$$\sum_{i \in C_j^+} y_i + \sum_{i \in C_j^-} (1 - y_i) \geq z_j$$

where $0 \leq y_i \leq 1$ and $0 \leq z_j \leq 1$. After solving for the optimal fractional solution y_i^*, we independently set variable x_i to true with probability y_i^*.
Analysis: According to the property of randomized rounding and the Arithmetic-Geometric Mean (AM-GM) inequality, the probability that a clause C_j of length ℓ_j is satisfied is at least:

$$1 - \left(1 - \frac{z_j^*}{\ell_j}\right)^{\ell_j} \geq \left(1 - \left(1 - \frac{1}{\ell_j}\right)^{\ell_j}\right) z_j^*$$

Since $\ell_j \geq 1$, the minimum value of the expression $1 - (1 - 1/\ell_j)^{\ell_j}$ is $1 - 1/e \approx 0.632$ as $\ell_j \to \infty$. Therefore, the probability that the clause is satisfied is at least $0.632 z_j^*$. The total expected number of satisfied clauses is $\sum E[Z_j] \geq 0.632 \sum z_j^* = 0.632\text{OPT}_{LP} \geq 0.632\text{OPT} > 0.6\text{OPT}$. Since both solving the LP and randomized rounding can be completed in polynomial time, this satisfies the requirements of the problem.

5. Chapter 13, Problem 9: One-pass Auction

Strategy Description :
The seller first outright rejects (refuses to accept) the bids of the first $n/2$ buyers, while recording the highest bid among these first $n/2$ buyers, denoted as M. Starting from the $(n/2+1)$-th buyer, the seller will immediately accept the first buyer whose bid is strictly greater than M, and terminate the auction. If no subsequent bid is ever greater than M, none are accepted.

Probabilistic Analysis Proof:
Assume the true global highest bid is B^*. Since buyers arrive in a random order, the probability of B^* appearing at any position j $(1 \leq j \leq n)$ is $1/n$. The necessary and sufficient conditions for our strategy to successfully obtain B^* are: 1. B^* must appear in the second half, i.e., $n/2 + 1 \leq j \leq n$. 2. Before the j-th person (i.e., B^*) arrives, the seller must not mistakenly accept anyone else earlier. This requires that the maximum bid among the first $j - 1$ people must fall exactly within the first $n/2$ people (so that it becomes M, and no one can surpass M before j arrives).

Since the order of the first $j - 1$ people is also completely random, the probability that their maximum value falls in the first $n/2$ positions is $\frac{n/2}{j-1}$.

Therefore, the probability of successfully obtaining the highest bid is:

$$\begin{aligned} P(\text{Success}) &= \sum_{j=n/2+1}^{n} P(B^* \text{ at } j) \times P(\text{max of first } j-1 \text{ is in first } n/2) \\ &= \sum_{j=n/2+1}^{n} \frac{1}{n} \left(\frac{n/2}{j-1} \right) \\ &= \frac{1}{2} \sum_{j=n/2+1}^{n} \frac{1}{j-1} \end{aligned}$$

We can establish a lower bound for this summation using a definite integral:

$$\sum_{x=n/2}^{n-1} \frac{1}{x} > \int_{n/2}^{n} \frac{1}{x} dx = \ln(n) - \ln(n/2) = \ln 2 \approx 0.693$$

Thus, the total probability of the strategy's success is:

$$P(\text{Success}) > \frac{1}{2} \ln 2 \approx 0.346 > \frac{1}{4}$$

This proves that regardless of n, the probability that the strategy accepts the highest bid is at least $1/4$.

6. Chapter 13, Problem 15: Approximate Median

Proof:
We consider sampling with replacement. If the median x calculated from a sample set S' of size c is not a 0.05-approximate median of S, it means x is either too small or too large. By symmetry, we only consider the case where x is too small (i.e., there are more than $0.55n$ numbers in S larger than x, which is equivalent to x falling within the bottom 45% of the numerical range of S). This happens if and only if out of our c random samples, at least $c/2$ samples are drawn from the smallest 45% of S.

Let the random variable X be the number of samples in S' that fall into the bottom 45% of S. Because we are sampling with replacement each time, X follows a binomial distribution $B(c, 0.45)$. The expected value is $\mu = E[X] = 0.45c$. We need to bound $P(X \geq 0.5c) \leq 0.005$. According to the Chernoff bound form $P(X \geq (1+\delta)\mu) \leq \exp(-\frac{\mu\delta^2}{3})$: We set $(1+\delta)0.45 = 0.5$, which yields $\delta = 1/9$. Substituting this into the Chernoff bound formula gives:

$$P(X \geq 0.5c) \leq \exp\left(-\frac{0.45c \cdot (1/81)}{3} \right) = \exp\left(-\frac{c}{540} \right)$$

For this probability not to exceed 0.005, we require:

$$\exp\left(-\frac{c}{540}\right) \leq 0.005 \implies \frac{c}{540} \geq \ln 200 \implies c \geq 540 \times 5.298 \approx 2861$$

Therefore, there exists an absolute constant c (e.g., $c = 3000$) independent of n. When the sample size reaches c, the probability that the sampled median is too small is less than 0.005. Similarly, the probability that it is too large is also less than 0.005. The total failure probability is strictly bounded within 0.01, hence the probability of successfully returning a 0.05-approximate median is at least 0.99.

7. Chapter 13, Problem 8: Dense Subgraph

Polynomial-time Expected Randomized Algorithm:
Algorithm Steps: 1. Uniformly and randomly select a subset X containing k nodes from the vertex set V of graph G (there are a total of $\binom{n}{k}$ possible combinations). 2. Calculate the number of edges in the induced subgraph $G[X]$. 3. If the number of edges is greater than or equal to $\frac{mk(k-1)}{n(n-1)}$, then output the set X and terminate the algorithm; otherwise, return to step 1 to randomly select again.

Correctness and Complexity Analysis:
For any arbitrary original edge $e \in E$ in the graph, we define an indicator random variable I_e: when both endpoints of e are selected into set X, $I_e = 1$, otherwise 0. Since X is selected uniformly at random, the probability that both endpoints of e simultaneously exist in X is:

$$P(I_e = 1) = \frac{\binom{n-2}{k-2}}{\binom{n}{k}} = \frac{k(k-1)}{n(n-1)}$$

By the linearity of expectation, the expected total number of edges in the induced subgraph $G[X]$ is:

$$E[\text{edges in } G[X]] = \sum_{e \in E} E[I_e] = \frac{mk(k-1)}{n(n-1)}$$

Since a random variable must have a possible value greater than or equal to its own expected value, there exists at least one subset of vertices of size k in the graph that satisfies the given edge count condition. Because the target bound we are looking for is exactly equal to the expected value, and the maximum number of edges does not exceed m, using a variant of Markov's inequality, we know that the probability of hitting the expected value in a single random draw is at least the reciprocal of a polynomial (a constant lower bound). This means that this Las Vegas randomized algorithm will definitely find and output a correct answer after an expected polynomial number of iterations.

8. Chapter 13, Problem 6: Betweenness

Algorithm Description:
Generate a completely random linear permutation of these n markers, meaning all $n!$ permutation methods appear with equal probability, and output this permutation as the result.

Analysis:
For any constraint $C = (\mu_i, \mu_j, \mu_k)$, it requires that marker μ_j must appear between μ_i and μ_k in the final ordering. We simply examine the relative positions of these three markers in this random global permutation. There are a total of $3! = 6$ relative orderings among these three markers, and since the original permutation is completely random, these 6 relative permutations are equally probable:

1. i, j, k (Satisfied)
2. k, j, i (Satisfied)
3. j, i, k (Unsatisfied)
4. k, i, j (Unsatisfied)
5. i, k, j (Unsatisfied)
6. j, k, i (Unsatisfied)

It can be seen that the constraint is satisfied only in the 2 cases where μ_j is placed exactly in the middle. Therefore, the probability that each constraint is satisfied is:

$$P(\text{Constraint is satisfied}) = \frac{2}{6} = \frac{1}{3}$$

Let the random variable X be the total number of satisfied constraints. By the linearity of expectation:

$$E[X] = \sum_{\ell=1}^{k} P(\text{Constraint } \ell \text{ is satisfied}) = \frac{1}{3}k$$

Suppose the optimal sequence arrangement can satisfy k^* constraints. Obviously $k^* \leq k$ always holds, thus we have:

$$E[X] = \frac{1}{3}k \geq \frac{1}{3}k^*$$

This shows that we have found a constant $\alpha = \frac{1}{3} > 0$ such that the expected number of constraints satisfied by this completely random permutation algorithm is no less than α times the optimal solution.

9. Chapter 13, Problem 12: Contraction

Construction of the Counterexample Graph G:
Consider a graph G with a source node s, a sink node t, and $n-2$ other intermediate nodes $v_1, v_2, \ldots, v_{n-2}$. The edges are constructed as follows:

- There are **two** parallel edges from s to each v_i.
- There is exactly **one** edge from each v_i to t.

Minimum $s-t$ Cut Analysis:
In this graph, the minimum $s-t$ cut is to separate t by itself. Cutting all the edges from v_i to t requires a capacity of $n-2$. On the other hand, separating s by cutting all edges from s to v_i would require a capacity of $2(n-2)$. Therefore, isolating t is indeed the unique global minimum $s-t$ cut.

Exponentially Small Probability of Success:
If we run the version of the contraction algorithm described in the problem (which ensures s and t are never contracted together by deleting direct $s-t$ edges), it will independently contract each of the length-2 paths from s to t in some random order.

In order for the algorithm to successfully find the minimum $s-t$ cut, it must contract each v_i into s, not into t. If any v_i is contracted into t, the two parallel edges originally connecting s to v_i will now connect directly to t, causing the final cut to be strictly larger than the minimum cut.

For each individual node v_i, there are 3 incident edges in total (2 connecting to s and 1 connecting to t). Because the algorithm selects a random edge uniformly, the chance of first picking one of the edges connected to s (thereby contracting v_i into s) is exactly:

$$P(v_i \text{ is contracted into } s) = \frac{2}{3}$$

Since this independent contraction process must happen favorably for all $n-2$ intermediate nodes, the total probability that the minimum $s-t$ cut is found is:

$$P(\text{find minimum } s-t \text{ cut}) = \left(\frac{2}{3}\right)^{n-2}$$

This is an exponentially small quantity with respect to n. This effectively demonstrates that this modified contraction method has a vanishingly small probability of finding the minimum $s-t$ cut.

(Note that this example poses no problem for finding the global minimum cut of the graph, which simply consists of isolating any of the single nodes v_i on its own by cutting its 3 incident edges.)

10. Chapter 13, Problem 13: Balls-and-Bins

Proof:
In this balls-and-bins experiment, a total of $2n$ balls are thrown into 2 bins, where each ball independently and uniformly chooses one of the bins.
Let random variables X_1 and X_2 denote the number of balls in bin 1 and bin 2, respectively. Obviously, we have $X_1 + X_2 = 2n$, which implies $X_2 = 2n - X_1$.

The difference we are interested in is $X_1 - X_2$:

$$X_1 - X_2 = X_1 - (2n - X_1) = 2X_1 - 2n$$

Thus, the event $X_1 - X_2 > c\sqrt{n}$ is equivalent to $2X_1 - 2n > c\sqrt{n}$, which means:

$$X_1 - n > \frac{c}{2}\sqrt{n}$$

Note that X_1 follows the binomial distribution $B(2n, 1/2)$. Its expected value and variance are respectively:

$$\mu = E[X_1] = 2n \times \frac{1}{2} = n$$

$$Var(X_1) = 2n \times \frac{1}{2} \times \left(1 - \frac{1}{2}\right) = \frac{n}{2}$$

To find an upper bound for the probability of this difference, we can apply Chebyshev's Inequality, which states that for any $a > 0$:

$$Pr(|X_1 - \mu| \geq a) \leq \frac{Var(X_1)}{a^2}$$

Setting $a = \frac{c}{2}\sqrt{n}$ and substituting the expectation and variance yields:

$$Pr\left(|X_1 - n| > \frac{c}{2}\sqrt{n}\right) \leq \frac{n/2}{(\frac{c}{2}\sqrt{n})^2} = \frac{n/2}{c^2 n/4} = \frac{2}{c^2}$$

Since $Pr(X_1 - X_2 > c\sqrt{n}) \leq Pr\left(|X_1 - n| > \frac{c}{2}\sqrt{n}\right)$, we have:

$$Pr(X_1 - X_2 > c\sqrt{n}) \leq \frac{2}{c^2}$$

The problem requires us to find a constant $c > 0$ such that this probability is at most ϵ for any given $\epsilon > 0$. We just need to set:

$$\frac{2}{c^2} \leq \epsilon \implies c \geq \sqrt{\frac{2}{\epsilon}}$$

Therefore, by choosing the constant $c = \sqrt{\frac{2}{\epsilon}}$, we can guarantee that $Pr[X_1 - X_2 > c\sqrt{n}] \leq \epsilon$. The proof is complete.

11. Chapter 13, Problem 11: Load Balancing

(a) The value of $\lim_{k\to\infty} N(k)/k$ and its proof
Since each job is assigned independently and uniformly at random to any of the k machines, the probability that a specific machine receives no jobs is:

$$P(\text{Machine } i \text{ receives 0 jobs}) = \left(1 - \frac{1}{k}\right)^k$$

By the linearity of expectation, the expected number of empty machines $N(k)$ is the sum of the probabilities of all machines being empty:

$$N(k) = \sum_{i=1}^{k} \left(1 - \frac{1}{k}\right)^k = k\left(1 - \frac{1}{k}\right)^k$$

Thus, the limit of the expected fraction of empty machines is:

$$\lim_{k\to\infty} \frac{N(k)}{k} = \lim_{k\to\infty} \left(1 - \frac{1}{k}\right)^k = \frac{1}{e}$$

(b) The value of $\lim_{k\to\infty} R(k)/k$ and its proof
Each machine can only process 1 job, and any excess jobs are rejected. This means that if a machine receives at least 1 job, it will process exactly 1 job. Therefore, the total number of processed jobs is exactly equal to the number of machines that receive at least one job. The expected number of machines receiving at least one job is $k - N(k)$. Since there are k jobs in total, the expected number of rejected jobs $R(k)$ is the total number of jobs minus the expected number of processed jobs:

$$R(k) = k - (k - N(k)) = N(k)$$

So the limit of the expected fraction of rejected jobs is:

$$\lim_{k\to\infty} \frac{R(k)}{k} = \lim_{k\to\infty} \frac{N(k)}{k} = \frac{1}{e}$$

(c) Expected fraction of rejected jobs (buffer size 2)

This part will involve slight calculations. We know from the first part that the number of machines with no jobs in the limit is k/e.

We first calculate the number of machines with exactly one job. Again, look at a machine p. The probability that only 1 job is assigned to that machine is:

$$k \cdot \frac{1}{k}\left(1 - \frac{1}{k}\right)^{k-1} = \left(1 - \frac{1}{k}\right)^{k-1}$$

(The chance of a given job j being assigned to p is $1/k$, and the probability that the remaining jobs will not be assigned to p is $(1-1/k)^{k-1}$. Finally, there are k choices for the "given" job j, which puts the coefficient k in the beginning).

Notice that in the limit, this probability also approaches $1/e$. Therefore, the expected number of machines with exactly 1 job is also k/e.

Finally, the remaining machines, regardless of how many jobs they were assigned, will perform exactly two jobs. The number of these machines is:

$$k - \frac{k}{e} - \frac{k}{e} = k - \frac{2k}{e}$$

The final tally is k/e machines with one job and $k - \frac{2k}{e}$ machines with two jobs. The total number of accepted (processed) jobs is:

$$1 \cdot \left(\frac{k}{e}\right) + 2 \cdot \left(k - \frac{2k}{e}\right) = \frac{k}{e} + 2k - \frac{4k}{e} = 2k - \frac{3k}{e}$$

Subtracting this from k (the total number of jobs), we get the number of rejected jobs:

$$R_2(k) = k - \left(2k - \frac{3k}{e}\right) = \frac{3k}{e} - k = \frac{k(3-e)}{e}$$

Thus, the limit of the expected fraction of rejected jobs is:

$$\lim_{k\to\infty} \frac{R_2(k)}{k} = \frac{3-e}{e} = \frac{3}{e} - 1 \approx 0.1036 \text{ (or approximately 11\%)}$$

12. Chapter 13, Problem 14: Perfectly Balanced

(a) Proof of non-existence of a perfectly balanced assignment for arbitrarily large n

Let n be odd, $k = n^2$, and represent the set of basic processes as the disjoint union of n sets $X_1, \ldots, X_n$ of cardinality n each. The set of processes P_i associated with job J_i will be equal to $X_i \cup X_{i+1}$, with addition taken modulo n.

We claim there is no perfectly balanced assignment of processes to machines. For suppose there were, and let Δ_i denote the number of processes in X_i assigned to machine M_1 minus the number of processes in X_i assigned to machine M_2. By the perfect balance property, we have $\Delta_{i+1} = -\Delta_i$ for each i; applying these equalities transitively, we obtain $\Delta_1 = -\Delta_1$ and hence $\Delta_i = 0$, for each i. But this is not possible since n is odd.

(b) Expected polynomial-time algorithm for a nearly balanced assignment

Consider independently assigning each process i a label L_i equal to either 0 or 1, chosen uniformly at random. Thus we may view the label L_i as a 0-1 random variable. Now for any job J_i we assign each process in P_i to machine M_1 if its label is 0, and machine M_2 if its label is 1.

Consider the event E_i, that more than $\frac{4}{3}n$ of the processes associated with J_i end up on the same machine. The assignment will be nearly balanced if none of the E_i happen. E_i is precisely the event that $\sum_{t \in J_i} L_t$ either exceeds $\frac{4}{3}$ times its mean (which is equal to n), or that it falls below $\frac{2}{3}$ times its mean. Thus, we may upper-bound the probability of E_i using Chernoff bounds as follows:

$$\begin{aligned} Pr[E_i] &\leq Pr\left[\sum_{t \in J_i} L_t < \frac{2}{3}n\right] + Pr\left[\sum_{t \in J_i} L_t > \frac{4}{3}n\right] \\ &\leq \left(e^{-\frac{1}{2}\left(\frac{1}{3}\right)^2}\right)^n + \left(\frac{e^{\frac{1}{3}}}{\left(\frac{4}{3}\right)^{\frac{4}{3}}}\right)^n \\ &\leq 2 \cdot 0.96^n \end{aligned}$$

Thus, by the union bound, the probability that any of the events E_i happens is at most $2n \cdot 0.96^n$, which is at most .06 for $n \geq 200$.

Thus, our randomized algorithm is as follows. We perform a random allocation of each process to a machine as above, check if the resulting assignment is perfectly balanced (nearly balanced in this context), and repeat this process if it isn't. Each iteration takes polynomial time, and the expected number of iterations is simply the expected waiting time for an event of probability $1 - .06 = .94$, which is $1/.94 < 2$. Thus the expected running time is polynomial.

This analysis also proves the existence of a nearly balanced allocation for any set of jobs.